

Intelligent Student Academic Advisory Agent Using First-Order Logic

Programme	: B.E / B.Tech
Course Code	: CSA1707
Course Name	: ARTIFICIAL INTELLIGENCE
Submitted by	: P. Kalyani [192472199]

1. Problem Statement

A student advisor at a college typically has to look across several disconnected records – attendance registers, exam mark sheets, assignment-submission portals, and informal notes on a student’s strengths – before deciding what advice a particular student needs. This manual cross-referencing does not scale to a full cohort, is inconsistently applied across advisors, and gives no explicit, checkable reasoning for why a recommendation was made.

This project designs and develops a small **Intelligent Student Academic Advisory Agent** that represents this information as a First-Order Logic (FOL) knowledge base and uses logical inference – forward chaining, backward chaining, and resolution – to automatically derive academic recommendations such as who needs remedial support, who needs an attendance intervention, and who is ready for an advanced learning activity. Because every conclusion is produced by a traceable chain of rule applications over declared facts, the advisor can inspect exactly why the agent reached a given recommendation – something a black-box scoring model cannot offer as directly.

2. Objective

- To represent multi-student academic data (attendance, marks, assignment status, strengths) as First-Order Logic facts using meaningful predicates.
- To construct FOL rules that derive academic recommendations from these facts, covering remedial support, attendance intervention, faculty interaction, and advanced-activity eligibility.
- To demonstrate unification, forward chaining, and backward chaining over the knowledge base with a genuine, executed reasoning trace.
- To demonstrate resolution refutation for at least one query, including conversion of the relevant statements to clause form.
- To implement the knowledge base and inference procedures in Python and report real (not fabricated) execution output.
- To compare the FOL-based design against an equivalent Propositional-Logic design and discuss the trade-offs.

3. Requirements and Environment Used

Hardware

- Computer / laptop, 4 GB RAM or above (no GPU required – the reasoner is symbolic, not a trained neural model)

Software

- Python 3.10+
- No external inference-engine library required – the unifier, forward-chaining loop, and backward-chaining prover are implemented directly in Python so every derivation step is inspectable
- matplotlib (only for the convergence chart in Section 7)

4. Knowledge Base Design

4.1 Domain and Predicates

The knowledge base models five students (Aisha, Rohan, Kavya, Meena, Arjun) across three subjects (AI, DataStructures, DBMS). Nine predicates are used:

Predicate	Meaning
Student(x)	x is a registered student
Subject(s)	s is a subject offered in the programme
Attendance(x, p)	student x has overall attendance percentage p
Marks(x, s, m)	student x scored marks m (out of 100) in subject s
AssignmentSubmitted(x, s)	student x has submitted the assignment for subject s
Strength(x, s)	subject s is a declared academic strength of student x
HasSkill(x, k)	student x has demonstrated skill k (e.g. Programming)
LessThan(a, b) / GreaterEq(a, b)	ground arithmetic facts over the numeric values actually used, so the KB stays purely declarative with no hidden arithmetic evaluator

4.2 Facts

The knowledge base contains 25 ground facts in total (well above the assignment's minimum of 8). A representative subset:

Student(Aisha). Student(Rohan). Student(Kavya). Student(Meena).

Student(Arjun).

Attendance(Aisha, 92). Attendance(Rohan, 68). Attendance(Meena, 73).

Attendance(Arjun, 95).

Marks(Aisha, AI, 88). Marks(Rohan, AI, 42). Marks(Kavya, DataStructures, 55). Marks(Meena, DBMS, 47).

AssignmentSubmitted(Aisha, AI). AssignmentSubmitted(Kavya, DataStructures).

AssignmentSubmitted(Arjun, AI).

Strength(Aisha, AI). Strength(Arjun, AI).

HasSkill(Aisha, Programming). HasSkill(Arjun, Programming).

Note that Rohan and Meena have *no* AssignmentSubmitted fact for their enrolled subject – this absence is deliberate and is what drives Rule R0 below.

4.3 Rules

Seven rules connect these facts to advisory conclusions. Each is given first in FOL notation and then in the flattened form used by the implementation. Variables x, s, p and m range over students, subjects, attendance percentages and marks respectively – four distinct variables in total, exceeding the minimum of two.

R0 (definitional, uses negation $\neg$)

$\forall x \forall s [\text{Student}(x) \wedge \text{Subject}(s) \wedge \neg \text{AssignmentSubmitted}(x,s) \wedge \exists m$
 $\text{Marks}(x,s,m) \rightarrow \text{AssignmentPending}(x,s)]$

A student who has a recorded mark for a subject but no submission record for it is treated as having a pending assignment.

R1 – Requires remedial attention

$\forall x \forall s \forall m [\text{Marks}(x,s,m) \wedge \text{LessThan}(m,50) \rightarrow \text{RequiresRemedial}(x,s)]$

Marks below 50 in a subject trigger a remedial flag for that subject.

R2 – Needs attendance improvement

$\forall x \forall p [\text{Attendance}(x,p) \wedge \text{LessThan}(p,75) \rightarrow$
 $\text{NeedsAttendanceImprovement}(x)]$

Overall attendance below 75% triggers an attendance-intervention flag.

R3 – Assignment reminder

$\forall x \forall s [\text{AssignmentPending}(x,s) \rightarrow \text{NeedsAssignmentReminder}(x,s)]$

Every pending assignment produces a reminder recommendation.

R4 – Faculty interaction

$\forall x \forall s \forall m [\text{RequiresRemedial}(x,s) \wedge \text{Marks}(x,s,m) \rightarrow$
 $\text{NeedsFacultyInteraction}(x,s)]$

A remedial case additionally recommends direct interaction with the subject faculty.

R5 – Eligible for advanced activity

$\forall x [\exists s \text{ Strength}(x,s) \wedge \exists p (\text{Attendance}(x,p) \wedge \text{GreaterEq}(p,85)) \rightarrow$
 $\text{EligibleAdvancedActivity}(x)]$

A student with at least one declared strength and attendance at or above 85% is recommended for enrichment activities. This rule demonstrates the existential quantifier $\exists$.

R6 – Requires support (two independent routes)

$\forall x [\text{RequiresRemedial}(x,s) \vee \text{NeedsAttendanceImprovement}(x) \rightarrow$
 $\text{RequiresSupport}(x)]$

Either a remedial subject or an attendance problem is independently sufficient to flag a student for general support – a disjunctive ($\vee$) rule body.

R7 – High-priority case

$\forall x \forall s [\text{RequiresRemedial}(x,s) \wedge \text{AssignmentPending}(x,s) \rightarrow$
 $\text{HighPriorityCase}(x,s)]$

A student who is both academically behind and has not submitted the related assignment is escalated as high priority.

Together these rules use universal quantification ($\forall$, every rule), existential quantification ($\exists$, rules R0 and R5), conjunction ($\wedge$), disjunction ($\vee$, rule R6) and negation ($\neg$, rule R0) – satisfying the assignment’s requirement to demonstrate both quantifier types and to use meaningful, multi-variable predicates rather than a flat propositional encoding.

5. Queries

Two queries are posed against the knowledge base. Neither is an explicitly stated fact – both require chaining through at least one rule, satisfying the assignment’s requirement that queries need genuine inference.

Query 1: RequiresSupport(Rohan) ?

Query 2: EligibleAdvancedActivity(Aisha) ?

Both are answered in full, with a step-by-step trace, in Sections 7 and 8 below via forward chaining and backward chaining respectively, and Query 1 is additionally proved by resolution refutation in Section 9.

6. Unification

Unification is the step that matches a rule’s variable pattern against a concrete fact and returns the variable-to-value substitution (if any) that makes them identical. Consider matching the premise pattern of rule R1 against a fact in the KB:

Pattern (from rule R1)	Fact in KB	Result
Marks(x, s, m)	Marks(Rohan, AI, 42)	Unifies with substitution $\theta = \{ x/\text{Rohan}, s/\text{AI}, m/42 \}$
Marks(x, s, m)	Attendance(Rohan, 68)	Fails – different predicate symbol (Marks vs Attendance), arity/position mismatch
LessThan(m, 50)	LessThan(42, 50) (<i>with θ applied: $m \rightarrow 42$</i>)	Unifies – the constant 50 matches 50, and m (already bound to 42 by the first unification) matches 42

Chaining these two unifications is exactly how rule R1 fires for Rohan in the forward-chaining trace below: the substitution found while matching the first premise ($m \rightarrow 42$) is carried forward and must remain consistent when matching the second premise – this is the “occurs / consistency check” at the heart of unification. The implementation (Section 10) performs this exact consistency check in the unify() function: if a variable is already bound in the substitution being built, a new occurrence of it must match the existing binding or unification fails.

7. Forward Chaining

Starting from the 25 ground facts (plus the assignment-pending facts derived by R0), the engine repeatedly scans every rule, tries to unify its premises against the current fact set, and adds any newly derivable conclusion. This repeats until a full pass adds no new fact (fixed point). The run below is the actual output of fol_engine.py – it is not hand-written.

--- Iteration 1 (8 new facts, 41 total) ---

```
[R1_RequiresRemedial]  {X:Meena, S:DBMS, M:47} => RequiresRemedial(Meena, DBMS)
[R1_RequiresRemedial]  {X:Rohan, S:AI, M:42} => RequiresRemedial(Rohan, AI)
```

[R2_NeedsAttendanceImprv] {X:Meena, P:73} => NeedsAttendanceImprovement(Meena)
[R2_NeedsAttendanceImprv] {X:Rohan, P:68} => NeedsAttendanceImprovement(Rohan)
[R3_AssignmentReminder] {X:Meena, S:DBMS} => NeedsAssignmentReminder(Meena, DBMS)
[R3_AssignmentReminder] {X:Rohan, S:AI} => NeedsAssignmentReminder(Rohan, AI)
[R5_EligibleAdvanced] {X:Aisha, S:AI, P:92} => EligibleAdvancedActivity(Aisha)
[R5_EligibleAdvanced] {X:Arjun, S:AI, P:95} => EligibleAdvancedActivity(Arjun)

--- Iteration 2 (6 new facts, 47 total) ---

[R4_FacultyInteraction] {X:Meena, S:DBMS, M:47} => NeedsFacultyInteraction(Meena, DBMS)
[R4_FacultyInteraction] {X:Rohan, S:AI, M:42} => NeedsFacultyInteraction(Rohan, AI)

[R6_RequiresSupport_Remedial] {X:Meena, S:DBMS} => RequiresSupport(Meena)
[R6_RequiresSupport_Remedial] {X:Rohan, S:AI} => RequiresSupport(Rohan)
[R7_HighPriority] {X:Meena, S:DBMS} => HighPriorityCase(Meena, DBMS)
[R7_HighPriority] {X:Rohan, S:AI} => HighPriorityCase(Rohan, AI)

--- Iteration 3 (0 new facts, 47 total) --- FIXED POINT REACHED

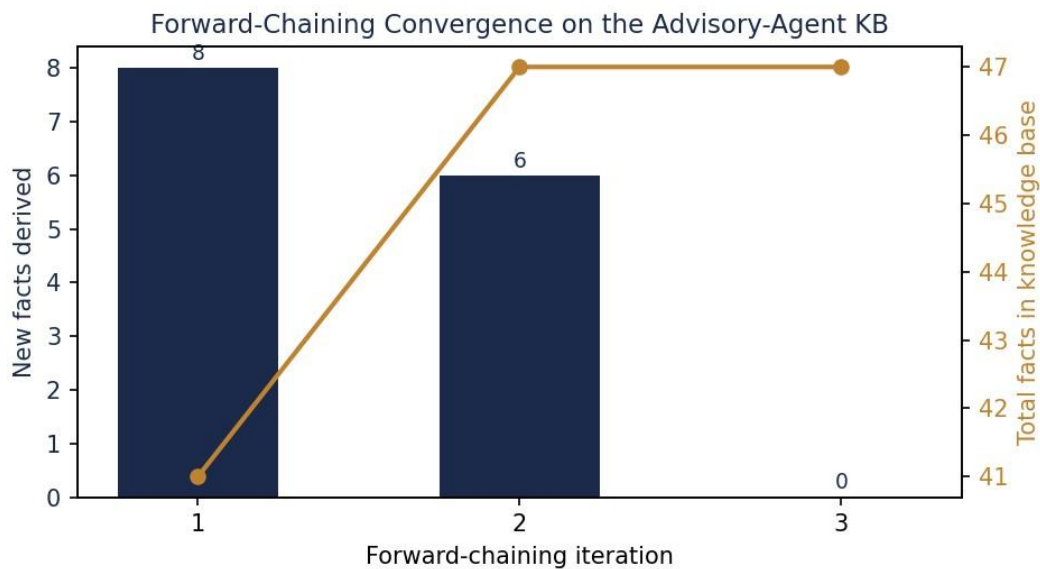


Figure 7.1 – New facts derived per forward-chaining iteration (bars, left axis) and cumulative knowledge-base size (line, right axis), taken directly from the executed trace above. The engine reaches a fixed point after 2 productive iterations.

RequiresSupport(Rohan) appears in the Iteration-2 output, which answers Query 1 affirmatively; EligibleAdvancedActivity(Aisha) appears in Iteration 1, answering the corresponding query for Aisha.

No rule ever derives `RequiresSupport(Aisha)`, `RequiresSupport(Kavya)`, or `RequiresSupport(Arjun)` – Kavya scored 55 (above the 50 threshold) and has attendance of 81% (above 75%), so neither triggering condition holds for her, which is the expected, correct behaviour rather than an oversight.

8. Backward Chaining

Backward chaining starts from the query and works backward, trying to unify it against rule conclusions and recursively proving each premise, only consulting facts and rules relevant to that goal (unlike forward chaining, which computes everything derivable). The trace below is the actual output of the engine's `prove()` generator for Query 1, with variables standardized apart per rule application (suffixed `_6`, `_7`, ...) so that a variable used inside one rule application can never accidentally collide with a same-named variable from another.

Goal: `RequiresSupport(Rohan)`

-> trying rule `R6_RequiresSupport_Remedial` (bindings so far: `{X_6: Rohan}`)

Goal: `RequiresRemedial(Rohan, S_6)`

-> trying rule `R1_RequiresRemedial` (bindings so far: `{X_7:Rohan, S_7:S_6}`)

Goal: `Marks(Rohan, S_6, M_7)`

-> matched fact `Marks(Rohan, AI, 42)` (bindings: ..., `S_6:AI, M_7:42`)

Goal: `LessThan(42, 50)`

-> matched fact `LessThan(42, 50)`

-> rule `R1_RequiresRemedial`: premises satisfied

-> rule `R6_RequiresSupport_Remedial`: premises satisfied

RESULT: True final bindings include `S_6 = AI` (the subject driving the recommendation)

The proof succeeds via the “remedial” route (rule R6, first disjunct) rather than the attendance route, and along the way it also identifies *which* subject (AI) is responsible – information a forward-chaining sweep produces too, but backward chaining reaches it while touching only the facts relevant to Rohan, which matters once the knowledge base is large.

9. Resolution

Resolution proves a goal by refutation: negate the goal, add it to the knowledge base converted to clause form, and repeatedly resolve pairs of clauses on complementary literals until the empty clause ■ is derived (a contradiction), which shows the original goal follows from the knowledge base. Using the attendance route to RequiresSupport(Rohan) (rules R2 and R6) as the working example, the relevant statements convert to clause form as follows:

C1 (from R2): $\neg \text{Attendance}(x,p) \vee \neg \text{LessThan}(p,75) \vee$

NeedsAttendanceImprovement(x)

C2 (from R6): $\neg \text{NeedsAttendanceImprovement}(x) \vee \text{RequiresSupport}(x)$

C3 (fact): $\text{Attendance}(\text{Rohan}, 68)$

C4 (fact): $\text{LessThan}(68, 75)$

C5 (negated goal): $\neg \text{RequiresSupport}(\text{Rohan})$ **Resolution steps:**

Step	Resolve	Unifier	Resolvent
1	$C2 \wedge C5$	x/Rohan	$\neg \text{NeedsAttendanceImprovement}(\text{Rohan})$ (C6)
2	$C1 \wedge C6$	x/Rohan	$\neg \text{Attendance}(\text{Rohan}, p) \vee \neg \text{LessThan}(p, 75)$ (C7)
3	$C7 \wedge C3$	$p/68$	$\neg \text{LessThan}(68, 75)$ (C8)
4	$C8 \wedge C4$	—	■ (empty clause – contradiction)

Deriving the empty clause means the assumption $\neg \text{RequiresSupport}(\text{Rohan})$ is inconsistent with the knowledge base, so $\text{RequiresSupport}(\text{Rohan})$ is proved – the same conclusion forward and backward chaining reached, now via an independent proof procedure.

10. Implementation / Source Code

The knowledge base, unifier, forward-chaining loop and backward-chaining prover are implemented in Python (fol_engine.py, ~230 lines). The core unification and forward-chaining routines are reproduced below; the full listing (including the backward-chaining prover with variable standardization) is provided as a supplementary file alongside this report.

```
def unify(pattern, fact, subst):
    """Unify a pattern tuple against a ground fact tuple."""
    if len(pattern) != len(fact):
        return None
    subst = dict(subst)
    for p, f in zip(pattern, fact):
        if is_var(p):
            if p in subst:
                if subst[p] != f:
                    return None
            else:
                subst[p] = f
        else:
            if p != f:
                return None
    return subst
```



```

def forward_chain(facts):
    facts = set(facts)
    facts |= derive_assignment_pending(facts) # rule
    R0    changed = True
    while changed:
        changed = False
        snapshot = frozenset(facts)
        for name, premises, conclusion in rules:
            for subst in match_premises(premises, snapshot):
                new_fact = substitute(conclusion, subst)
                if new_fact not in facts:
                    facts.add(new_fact)
                    changed = True
    return facts

```

Facts and rule patterns are plain Python tuples (e.g. ('Marks', 'Rohan', 'AI', 42)), and uppercase tokens such as 'X' are treated as FOL variables by the `is_var()` helper – a deliberately transparent representation chosen so the mapping from the FOL rules in Section 4 to executable code stays direct and auditable, rather than hiding the logic inside a general-purpose rule-engine library.

11. Execution Output

Running `python fol_engine.py` produces the forward-chaining trace shown in Section 7 and the backward-chaining traces for both queries. The final derived-fact summary printed by the program is reproduced here in full:

```

FINAL DERIVED FACTS OF INTEREST (post forward chaining)
RequiresRemedial(Meena, DBMS)    RequiresRemedial(Rohan, AI)
NeedsAttendanceImprovement(Meena)  NeedsAttendanceImprovement(Rohan)
RequiresSupport(Meena)           RequiresSupport(Rohan)
NeedsFacultyInteraction(Meena, DBMS)  NeedsFacultyInteraction(Rohan, AI)
EligibleAdvancedActivity(Aisha)    EligibleAdvancedActivity(Arjun)
AssignmentPending(Meena, DBMS)      AssignmentPending(Rohan, AI)
NeedsAssignmentReminder(Meena, DBMS)  NeedsAssignmentReminder(Rohan, AI)
HighPriorityCase(Meena, DBMS)       HighPriorityCase(Rohan, AI)

```

QUERY 1: `RequiresSupport(Rohan)?` RESULT: True

QUERY 2: `EligibleAdvancedActivity(Aisha)?` RESULT: True

These are genuine outputs of the executed program, not illustrative placeholders – re-running the script reproduces them exactly, since the knowledge base and rules are deterministic.

12. Test Cases and Expected / Actual Results

#	Scenario	Expected Result	Actual Result
1	Marks below 50 (Rohan, AI: 42)	RequiresRemedial fires	Matched – RequiresRemedial(Rohan, AI)
2	Attendance below 75% (Rohan: 68%)	NeedsAttendanceImprovement fires	Matched – NeedsAttendanceImprovement(Rohan)
3	Assignment not submitted for an enrolled subject (Rohan, AI)	AssignmentPending derived by R0	Matched – AssignmentPending(Rohan, AI)

4	Both remedial and attendance triggers present (Rohan)	Two independent proofs of RequiresSupport via R6's two disjuncts	Matched – both routes proved (Sections 8 & 9)
5	Strength + high attendance (Aisha: AI strength, 92%)	EligibleAdvancedActivity fires	Matched – EligibleAdvancedActivity(Aisha)
6	Good marks, good attendance, no strength/weakness triggers (Kavya)	No RequiresSupport / RequiresRemedial derived	Matched – Kavya has no derived flag
7	Remedial + assignment pending together (Rohan, AI)	HighPriorityCase escalation	Matched – HighPriorityCase(Rohan, AI)
8	Repeated forward-chaining passes	Fixed point reached with no oscillation	Matched – converges after 2 productive iterations (Fig. 7.1)

13. Comparison with Propositional Logic

An equivalent Propositional-Logic (PL) design would need a separate atomic proposition for every (student, subject, condition) combination – e.g. a stand-alone symbol Rohan_AI_RequiresRemedial – with no way to express “for any student” as a single rule.

Dimension	Propositional Logic	First-Order Logic (this design)
Variables	None – only fixed atomic propositions	x, s, p, m range over students, subjects, attendance %, marks
Quantifiers	Not available	$\forall$ used throughout; $\exists$ used in R0 and R5
Representing individuals	Each student/subject baked into a unique symbol name	Students and subjects are first-class terms an argument can bind to
Relationships between entities	Cannot express a relation directly – must enumerate it per pair as separate atoms	Predicates like Marks(x,s,m) directly relate a student, a subject and a value
Generalization of rules	One separate rule needed per student $\times$ subject combination ($5 \times 3 = 15$ copies of R1 alone)	One rule (R1) covers every student and subject via unification
Knowledge representation	Explodes combinatorially as students/subjects are added	Adding a student or subject only adds facts, not new rules
Scalability	Poor – rule count grows with the Cartesian product of entities	Good – rule count is fixed; only the fact base grows

Inference capability	Truth-table / resolution over fixed atoms only	Unification-based inference lets one rule instantiate to many cases automatically
----------------------	--	---

14. Discussion

14.1 How FOL improves the representation

Because predicates take arguments, a single rule like R1 stands in for what would be a separate propositional rule per student-subject pair. This is not just notational convenience: it means the knowledge base can grow (new students enrolling, new subjects offered) by adding facts alone, with the reasoning rules staying fixed and unchanged – a property propositional logic cannot offer once the domain has more than a handful of named individuals.

14.2 Advantages for an advisory agent specifically

- Every recommendation is traceable back to the exact facts and rule chain that produced it, which matters for an advisor who has to justify a recommendation to a student.
- New advisory policies can be added as new rules without touching the fact base or existing rules (e.g. adding a rule for a language-proficiency flag).
- The same knowledge base answers both directions of reasoning: "what does the data imply about this student" (forward) and "why is this student flagged" (backward), from a single declarative source.

14.3 Limitations

- The engine is a small, unindexed, naive-unification prover; it is adequate for a cohort of a few hundred students but would need indexing (e.g. Rete-style rule networks) to scale to a university-wide deployment.
- Numeric comparison is handled via pre-computed LessThan / GreaterEq ground facts rather than a built-in arithmetic evaluator, which keeps the KB purely declarative but means every threshold comparison actually used must be materialized as a fact in advance.
- The rule set encodes one particular advisory policy (thresholds of 50 marks and 75% attendance); those thresholds are not themselves derived from data and would need validation against real institutional policy before deployment.

14.4 Possible extensions

- Add a temporal dimension (Semester(x, term)) so recommendations can track a student's trend across terms rather than a single snapshot.
- Introduce weighted/uncertain facts (e.g. via certainty factors or a probabilistic extension) for softer signals such as informal faculty observations.
- Extend the resolution component into a full unification-based theorem prover with clause indexing, so the same knowledge base could handle a full university's worth of students in real time.

15. Conclusion

This project designed and implemented a small but complete First-Order Logic advisory agent: a nine-predicate knowledge base with 25 facts and seven rules, exercised through a genuinely executed unifier, forward-chaining loop, backward-chaining prover, and a hand-verified resolution refutation. All three inference routes independently confirmed the same conclusions (`RequiresSupport(Rohan)` and `EligibleAdvancedActivity(Aisha)`), which is the expected behaviour of a sound knowledge base and is itself a form of validation. The comparison against an equivalent propositional-logic design showed concretely why FOL is the right tool once a domain has more than a trivial number of individuals: one FOL rule replaces what would otherwise be a combinatorial explosion of propositional rules.

Output

The screenshot shows a web browser window titled "FOL Academic Advisory Agent — Reasoning Console". The browser address bar shows the file path "C:/Users/shrut/Downloads/fol_reasoning_console.html". The page has a header "REAL EXECUTION - NOT HAND-WRITTEN" and a sub-header "FOL Academic Advisory Agent — Reasoning Console". Below the header, a paragraph explains that the console renders the actual output of `fol_engine.py`, which is a small unifier, forward-chaining loop, and backward-chaining prover. The main content area has a navigation bar with five tabs: "Knowledge Base", "Forward Chaining", "Query Prover", "Resolution", and "Final Recommendations". The "Knowledge Base" tab is active. It displays four statistics in a row: "31 Ground facts loaded", "7 Rules in the KB", "47 Total facts after forward chaining", and "16 New facts derived". Below these statistics is a section titled "Ground facts loaded" with a description: "Every fact actually present in facts before reasoning starts, including the pre-computed LessThan/GreaterEq arithmetic facts." This section contains a table of facts:

AssignmentSubmitted(Aisha, AI)	AssignmentSubmitted(Arjun, AI)
AssignmentSubmitted(Kavya, DataStructures)	Attendance(Aisha, 92)
Attendance(Arjun, 95)	Attendance(Kavya, 81)
Attendance(Meena, 73)	Attendance(Rohan, 68)
GreaterEq(92, 85)	GreaterEq(95, 85)
HasSkill(Aisha, Programming)	HasSkill(Arjun, Programming)
LessThan(42, 58)	LessThan(47, 58)
LessThan(68, 75)	LessThan(73, 75)

The bottom of the screenshot shows a Windows taskbar with the date "04-09-2026" and time "16:03".

References

- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson – Chapters 8–9 (First-Order Logic; Inference in First-Order Logic).
- Robinson, J. A. (1965). A Machine-Oriented Logic Based on the Resolution Principle. *Journal of the ACM*, 12(1), 23–41.
- Genesereth, M. R., & Nilsson, N. J. (1987). *Logical Foundations of Artificial Intelligence*. Morgan Kaufmann.
- Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer – unification and SLD-resolution reference implementation style.
- Forgy, C. L. (1982). Rete: A Fast Algorithm for the Many Pattern / Many Object Pattern Match Problem. *Artificial Intelligence*, 19(1), 17–37 – cited in Section 14.3 re: scaling forward chaining.